Count Vectorizer :- Imp

Purpose :- To convert text (words) into numerical (matrix) features that a machine learning model can understand.

Eg:- Suppose we have 4 email messages (documents):

Doc 1 :- Hey, how are you
Doc 2 :- Win money now
Doc 3 :- Hey, win the game now
Doc 4 :- Are you going to the game

Step 1 - Find all unique words (vocabulary)

Combine all words from all documents & remove duplicates.

["hey", "how", "are", "you", "win", "money", "now", "the", "game", "going", "to"]

There are 11 unique words → these become columns (features)

Step2:- Count occurence of each word in every

Document	hey	how	are	you	win	money	now	the
Doc1	1	1	1	1	0	0	0	0
Doc2	0	0	0	0	1	1	1	0
Doc3	1	0	0	0	1	0	1	1
Doc4	0	0	1	1	0	0	0	1

Each number shows how many times a word appears in that document.

Step3:- Use this matrix as features

- Each row = one email (document)
- Each column = one unique word (feature)
- Each value = how often that word appears in that email.

This numeric matrix is given to the Naive Bayes model for training.

Key points:-

- Naive Bayes works well for spam detection
- CountVectorizer converts text into numerical features
- pipeline simplifies preprocessing + modeling
- Multinomial NB is best for discrete text data.

A pipeline is a tool in scikit-learn that helps us combine multiple data preprocessing & a machine learning model into a single workflow.

It ensures that all steps are executed in the correct order, making the process clean, reproducible, & error-free.

Purpose:-

- To organize the workflow & keep code clean.
- To avoid repeating transformations on training & testing data.
- To prevent data leakage → (applying test data transformations incorrectly).
- To enable easy parameter tuning with GridSearchCV
- To allow consistent handling of both training & prediction.

Concept, Imp

- A pipeline is a sequence of steps, each step represented as (tuple)

  ('step_name', transformer or estimator)

- Step name (string) → is any label/name you choose. These names are useful, when we want to refer to a specific step

  Eg:-

  ('brain', MultinomialNB()),
  ('text_handler', CountVectorizer())

  It can be anything meaningful, like 'vectorizer', 'tfidf', 'classifier', etc.

- Transformer or estimator is the actual object like CountVectorizer(), TfidfVectorizer(), StandardScaler(), or MultinomialNB().

Important Points -

1. Order matters

→ each output's step is input for the next step!

- Transformers (like CountVectorizer) must come first.
- Estimators/model (like MultinomialNB) must come after that or at last).

2. The transformers (like CountVectorizer, StandardScaler, etc) act only on (X) (all early steps).
ie. X (features) flows through all steps of pipeline.
And the (final estimator step)/model (like NaiveBayes, LogisticRegression, etc) is the one that actually uses both (X and y). (y (labels, target))

(fit) only once on the training data;
then use the same pipeline to transform test data.

Eg: -

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import
    CountVectorizer
from sklearn.naive_bayes import MultinomialNB

spam_model = Pipeline([                    # step1: Transform text → numeric
    ('vectorizer', CountVectorizer()),
    ('classifier', MultinomialNB())        # step2: Train model on numeric features
])

spam_model.fit(X_train, y_train)

accuracy = spam_model.score(X_test, y_test)
print("Accuracy:", accuracy)
```

How it Works

- Training -

1. (X_train) → step1 transformer → step2 transformer → ... → final model we (transformed X_train) + (y_train)

- Prediction:-
  1- [X_test] → all transformers in order
  ↓
  final model predicts [y_pred]

Advantages

- Reduces manual preprocessing
- Prevents human errors
- Keeps transformations consistent between training & testing
- Integrates easily with parameter tuning (GridSearchCV)
- Single interface for fit(), predict(), score().

[Email Text]
↓
[Dataset: category + message]
↓
[Convert category to numbers]
ham → 0
spam → 1
↓
[Train-Test split]
X_train, X_test, y_train, y_test
↓
[Text → features (CountVectorizer)]
Each unique word → a column
Word frequency → value in matrix
↓
[Train Multinomial Naive Bayes Model]
model.fit(X_train_count, y_train)
↓
[predictions]
model.predict(X_test_count)
↓
[Evaluation]
Accuracy ~ 98%.

[optional: pipeline]
automatically converts text → vectors → NB → prediction